Reescrita de Consultas SQL Baseada em Modelos de Linguagem de Grande Escala (LLM-based SQL Query Rewriting)

Luiz Eduardo Treméa[1], **Fahad Kalil**[1]

[1]Ciência da Computação – Atitus Educação
Passo Fundo – RS – Brasil

11123471@atitus.edu.br, fahad.kalil@atitus.edu.br

Abstract. *Escreva seu resumo em língua estrangeira (inglês)...*

Resumo. *Resumo do trabalho (português)...*

1. Introdução

Cenários, motivação, justificativa, problema de pesquisa, objetivo geral e específicos (taxonomia de Bloom);

2. Referencial Teórico

Nesta seção, são apresentados os fundamentos teóricos que sustentam o uso de Inteligência Artificial e Modelos de Linguagem de Grande Escala (*Large Language Models – LLMs*) para a otimização e reescrita de consultas em sistemas de banco de dados.

2.1. Evolução da Otimização de Consultas e Uso de Inteligência Artificial

A evolução da otimização de consultas reflete a transição de um paradigma puramente baseado em modelos matemáticos estáticos para sistemas adaptativos e centrados nos dados (**ref_otimizacao_geral**). Tradicionalmente, os otimizadores baseados em custo utilizam três componentes fundamentais: *Cardinality Estimation* (CE), *Cost Model* (CM) e *Plan Enumeration* (PE) (**ref_ce_cm_pe**). O componente CE utiliza estatísticas como histogramas unidimensionais, amostragem e *sketches* para prever o número de tuplas processadas por cada operador (**ref_histogramas**). O modelo de custo, por sua vez, é definido por equações artesanais que combinam CPU, I/O e memória para atribuir um valor escalar a cada plano candidato (**ref_cost_model**).

Apesar de consolidadas, essas abordagens apresentam limitações importantes em *workloads* modernos (**ref_limitacoes_otimizadores**). O principal problema reside na suposição de independência entre atributos e distribuições uniformes, frequentemente incompatíveis com dados reais correlacionados ou enviesados (*skewed*) (**ref_correlated_data**). Erros na estimativa de cardinalidade propagam-se ao longo das árvores de execução e podem conduzir o otimizador à seleção de planos significativamente inferiores (**ref_cardinality_errors**). Além disso, o ajuste manual realizado por especialistas (*Database Administrators* – DBAs) possui alto custo operacional e baixa escalabilidade em ambientes distribuídos e de nuvem (**ref_manual_tuning**).

Nesse contexto surge a *Learned Query Optimization* (LQO), cuja principal diferença em relação à otimização clássica está na forma de representação do conhecimento (**ref_lqo**). Enquanto abordagens heurísticas dependem de regras fixas e estáticas, a LQO incorpora modelos de *Machine Learning* capazes de aprender relações entre características da consulta e desempenho real observado em execuções anteriores (**ref_heuristics_vs_ml**).

Dois sistemas destacam-se nessa transição:

- **Neo (*Neural Optimizer*)**
  O Neo é um otimizador ponta a ponta que substitui modelos de custo tradicionais por uma *Value Network* baseada em redes neurais convolucionais em árvore (*Tree Convolutions*) (**marcus2019neo**). O sistema utiliza Aprendizado por Reforço (*Reinforcement Learning*) e estratégias de *best-first search* para prever o plano de execução mais eficiente, refinando continuamente suas decisões por meio do feedback de latência observado (**marcus2019neo**).
- **Bao (*Bandit Optimizer*)**
  Diferentemente do Neo, o Bao atua como uma camada complementar ao otimizador existente, fornecendo *hints* de execução ao mecanismo tradicional (**marcus2021bao**). O problema é modelado como um *Contextual Multi-Armed Bandit*, utilizando *Thompson Sampling* para selecionar o conjunto de dicas com maior probabilidade de sucesso para cada consulta (**marcus2021bao**). O Bao destaca-se pela eficiência de treinamento e pela adaptação dinâmica a mudanças de esquema e dados.

Essas limitações associadas à rigidez heurística e à incapacidade de modelar relações complexas entre dados impulsionaram o avanço de soluções baseadas em Inteligência Artificial (**ref_limitacoes_lqo**).

2.2. SQL Query Rewriting via LLMs (SQL-to-SQL)

O conceito de *SQL Query Rewriting via Large Language Models* (LLMs) envolve transformar uma instrução SQL em uma versão logicamente equivalente, porém mais eficiente, utilizando as capacidades de compreensão estrutural e raciocínio dos modelos de linguagem (**ref_sql_rewriting**). Diferentemente de sistemas *Text-to-SQL*, cujo objetivo é converter linguagem natural em SQL, a reescrita *SQL-to-SQL* opera diretamente sobre consultas formais já existentes, tornando a equivalência semântica um requisito indispensável (**ref_sql_to_sql**).

As transformações mais recorrentes na literatura incluem o uso de *Common Table Expressions* (CTEs), decorrelação de subconsultas, reordenação de junções, *predicate pushdown*, pré-agregação e simplificação lógica (**ref_transformacoes_sql**). Essas técnicas visam reduzir redundâncias, minimizar processamento intermediário e racionalizar operações de alto custo computacional (**ref_cte_pushdown**).

Para garantir a correção da reescrita, os sistemas utilizam mecanismos de validação em múltiplos estágios (**ref_semantic_validation**). A validação sintática é geralmente realizada pelo próprio *parser* do SGBD, permitindo a autocorreção do modelo (**ref_parser_validation**). Já a equivalência semântica pode ser verificada por ferramentas baseadas em lógica formal e *Satisfiability Modulo Theories* (SMT), ou ainda pela comparação dos resultados obtidos sobre conjuntos de dados amostrais (**ref_qed_sqlsolver**).

Os principais riscos dessas abordagens incluem alucinações, *semantic drift*, falhas sintáticas e instabilidade dos planos gerados (**ref_llm_risks**). O papel dos LLMs varia conforme a arquitetura proposta.

- **LLM-R2**
  Utiliza *In-Context Learning* para recomendar sequências de regras derivadas do Apache Calcite, restringindo o espaço de busca e reduzindo erros semânticos (**li2024llmr2**).
- **R-Bot**
  Emprega arquitetura baseada em *Retrieval-Augmented Generation* (RAG), recuperando documentação técnica e histórico de consultas para fundamentar a geração da reescrita (**sun2025rbot**).
- **QUITE**
  Baseia-se em uma arquitetura multiagente controlada por Máquina de Estados Finita (*Finite State Machine* – FSM), refinando iterativamente as consultas com feedback em tempo real do banco (**song2025quite**).
- **LITHE / AGENTIC-LITHE**
  Exploram transformações diretamente no espaço da consulta, utilizando telemetria probabilística e algoritmos como *Monte Carlo Tree Search* (MCTS) para explorar o espaço de reescritas (**ref_lithe**).

O desempenho dessas abordagens é normalmente avaliado por métricas como redução de latência, custo estimado pelo otimizador, taxa de equivalência e percentual de melhoria efetiva de desempenho (**ref_metrics_sql_rewrite**).

2.3. Estimativa de Cardinalidade e Modelos Aprendidos

A estimativa de cardinalidade é frequentemente descrita como o "calcanhar de Aquiles" da otimização de consultas (**ref_cardinality_problem**). Sua função consiste em prever o número de tuplas resultantes de cada operador em um plano de execução, permitindo que o modelo de custo estime o consumo de recursos computacionais (**ref_ce_function**). Erros nessa etapa propagam-se ao longo das árvores de junção e podem produzir escolhas inadequadas de operadores físicos, como a seleção incorreta entre *Nested Loop Join* e *Hash Join* (**ref_join_errors**).

Para superar as limitações das estatísticas tradicionais, surgiram modelos aprendidos orientados a dados.

- **NeuroCard**
  Propõe um estimador unificado para todo o banco de dados baseado em modelos autorregressivos profundos (**yang2021neurocard**). O sistema aprende a distribuição probabilística da junção externa total das tabelas, capturando correlações complexas sem assumir independência estatística (**yang2021neurocard**). Técnicas de fatoração sem perdas permitem lidar com colunas de alta cardinalidade mantendo eficiência computacional.
- **FLAT**
  Introduz as *Factorize-Split-Sum-Product Networks* (FSPN), capazes de decompor a distribuição conjunta de forma adaptativa (**zhu2021flat**). O modelo diferencia atributos fortemente e fracamente correlacionados, conciliando precisão elevada com baixo custo de inferência e armazenamento (**zhu2021flat**).

Em comparação com histogramas tradicionais, esses modelos apresentam vantagens expressivas ao capturar correlações de alta ordem e distribuições enviesadas (**ref_histogram_limits**). Trabalhos recentes apontam reduções significativas no *q-error* em *benchmarks* complexos como o JOB-light (**ref_joblight**).

Existe ainda uma relação direta entre estimativa de cardinalidade e reescrita via LLMs. Sistemas avançados utilizam informações estatísticas provenientes do otimizador para orientar transformações semanticamente equivalentes e estatisticamente favoráveis (**ref_lithe_cardinality**). Estimativas precisas fornecem contexto factual sobre densidade e seletividade dos dados, reduzindo o risco de regressão de desempenho decorrente da reescrita (**ref_selectivity_guidance**).

2.4. Arquiteturas RAG e Sistemas Baseados em Agentes

A *Retrieval-Augmented Generation* (RAG) aplicada a bancos de dados busca reduzir alucinações em LLMs ao incorporar conhecimento externo e evidências específicas do domínio durante a geração ou reescrita de consultas (**ref_rag_db**). No contexto SQL, essa abordagem permite recuperar estratégias previamente validadas e documentação especializada que complementam o conhecimento interno do modelo (**ref_rag_sql**).

O processo geralmente envolve duas etapas principais: recuperação e fundamentação (*grounding*) (**ref_rag_pipeline**). Inicialmente, consultas e documentos são convertidos em representações vetoriais (*embeddings*), permitindo busca híbrida baseada em semântica e estrutura (**ref_embeddings**). Posteriormente, os artefatos recuperados são utilizados como receitas de reescrita e contexto operacional para orientar a decisão do modelo (**ref_grounding**).

Arquiteturas multiagente expandem esse paradigma ao distribuir responsabilidades entre componentes especializados.

- **Planner**
  Responsável por analisar a estrutura da consulta e propor estratégias iniciais de otimização (**ref_planner_agent**).
- **Reviewer**
  Executa validação sintática e semântica, utilizando compiladores SQL e verificadores formais (**ref_reviewer_agent**).
- **Executor**
  Aplica as transformações aprovadas e coleta métricas reais de desempenho do banco de dados (**ref_executor_agent**).

O elemento central dessas arquiteturas é o *feedback loop* (**ref_feedback_loop**). Métricas derivadas de comandos como `EXPLAIN` e `EXPLAIN ANALYZE` permitem comparar custo estimado e execução real, alimentando processos iterativos de autocorreção (**ref_explain_feedback**). Quando uma reescrita não produz o ganho esperado ou viola restrições semânticas, o sistema reinicia o ciclo de decisão incorporando o histórico da falha (**ref_self_correction**). Essa dinâmica aproxima a reescrita baseada em LLMs de sistemas adaptativos orientados por evidências operacionais reais.

2.5. Comparação Crítica, Limitações e Oportunidades de Pesquisa

A análise comparativa entre sistemas de reescrita de consultas baseados em *Large Language Models* (LLMs) e abordagens de *Learned Query Optimization* (LQO) evidencia

uma mudança progressiva de paradigma na otimização de consultas SQL (**ref_paradigm_shift**). Enquanto os otimizadores aprendidos concentram-se predominantemente na melhoria interna dos mecanismos de planejamento e execução, os sistemas de *SQL-to-SQL rewriting* deslocam o foco para transformações semânticas realizadas na camada de interface declarativa da consulta (**ref_lqo_vs_rewrite**). Essa distinção altera significativamente os mecanismos de decisão, os requisitos de validação e o grau de intervenção sobre o motor de banco de dados.

2.5.1. Limitações e Barreiras dos Sistemas Atuais

Apesar dos avanços recentes, a literatura aponta desafios relevantes que ainda restringem a adoção ampla de reescritores baseados em LLMs em ambientes corporativos (**ref_limitations_llm**).

Um dos principais obstáculos está relacionado à confiança e à verificação da equivalência semântica (**ref_semantic_bottleneck**). Diferentemente de aplicações *Text-to-SQL*, nas quais certo grau de ambiguidade é tolerável, a reescrita SQL exige preservação absoluta da lógica original da consulta. Verificadores formais baseados em lógica ou *Satisfiability Modulo Theories* (SMT) apresentam cobertura limitada para consultas envolvendo agregações complexas, múltiplas junções ou subconsultas aninhadas (**ref_smt_limits**). Já mecanismos de validação por execução podem produzir falsos positivos quando os dados utilizados nos testes não contemplam todas as combinações relevantes de predicados (**ref_execution_validation_limit**).

Outro fator crítico refere-se ao custo computacional e à latência de inferência (**ref_inference_cost**). Enquanto otimizadores tradicionais operam em escala de milissegundos, modelos de linguagem frequentemente demandam segundos para produzir uma resposta (**ref_latency_gap**). Em cenários de produção, o custo financeiro associado ao uso contínuo de APIs comerciais torna inviável a reescrita indiscriminada de todas as consultas, motivando o desenvolvimento de mecanismos de roteamento e seleção dinâmica de modelos (*model routing*) (**ref_model_router**).

A literatura também destaca problemas relacionados a alucinações e *semantic drifting* (**ref_hallucinations**). Em determinadas situações, modelos podem sugerir regras inexistentes ou alterar inadvertidamente a lógica da consulta original, produzindo resultados incorretos ou regressões severas de desempenho (**ref_semantic_drift**). Além disso, tanto LLMs quanto otimizadores aprendidos frequentemente operam como caixas-pretas, reduzindo a capacidade de auditoria e dificultando a investigação de regressões por administradores de banco de dados (**ref_blackbox_problem**).

2.5.2. Comparação Técnica entre LQO e Sistemas de Reescrita via LLM

A Tabela 1 sintetiza as diferenças arquiteturais entre os principais trabalhos da literatura.

A comparação evidencia diferenças substanciais entre as abordagens (**ref_compare_systems**). Neo e Bao concentram-se no interior do otimizador. O Neo propõe uma reconstrução profunda do modelo de custo e da seleção de planos, enquanto o Bao preserva a estrutura do otimizador tradicional e apenas orienta suas escolhas por meio de *hints* físicos

Tabela 1. Comparação entre sistemas de LQO e reescrita baseada em LLMs

Sistema	Espaço de Transformação	Mecanismo de Decisão	Uso de IA/LLM	Feedback e Resiliência
Neo	Espaço de plano	*Value Network* (Tree-CNN)	*Deep Learning* ponta-a-ponta	*Bootstrapping* via PostgreSQL
Bao	Conjunto de *hints*	*Thompson Sampling*	Aprendizado por reforço	Adaptativo ao *workload*
LLM-R2	Seleção de regras	*In-Context Learning*	LLM + modelo contrastivo	Equivalência via Calcite
R-Bot	Seleção de regras	RAG + raciocínio passo-a-passo	LLM + recuperação híbrida	Auto-reflexão e *recipes*
QUITE	Espaço de consulta	Multiagentes + FSM	LLM agents + MDP	Feedback em tempo real
LITHE	Espaço de consulta	MCTS sobre tokens	LLM + telemetria	Validação lógica e estatística

(**marcus2019neo**; **marcus2021bao**). Em ambos os casos, entretanto, não há atuação direta sobre a estrutura declarativa da consulta.

Em contraste, LLM-R2 e R-Bot operam como orquestradores externos de regras e conhecimento especializado (**li2024llmr2**; **sun2025rbot**). O LLM-R2 restringe o espaço de decisão a regras previamente conhecidas, reduzindo riscos semânticos, enquanto o R-Bot introduz o uso de *Retrieval-Augmented Generation* para recuperar receitas de otimização derivadas de documentação técnica e experiências anteriores.

Sistemas mais recentes, como QUITE e LITHE, apresentam maior independência de regras fixas (**song2025quite**; **ref_lithe**). O QUITE utiliza arquitetura multiagente para iterar sobre a consulta até que o feedback operacional do banco indique melhoria efetiva de desempenho. Já o LITHE emprega *Monte Carlo Tree Search* sobre probabilidades de geração de tokens, explorando alternativas de reescrita em regiões onde o modelo apresenta menor confiança estatística.

2.5.3. Lacunas Científicas e Oportunidades de Pesquisa

Embora os avanços recentes tenham ampliado significativamente o potencial da reescrita automática de consultas, diversas lacunas permanecem abertas (**ref_research_gaps**).

Uma primeira limitação refere-se ao chamado *impedance mismatch* semântico (**ref_impedance**). Existe um desalinhamento entre as transformações propostas pelos modelos de linguagem e as capacidades efetivamente suportadas pelos motores de execução atuais. Muitas reescritas semanticamente sofisticadas não podem ser traduzidas diretamente em regras genéricas de otimização.

Outra oportunidade de pesquisa envolve o aprendizado automático de regras de transformação (**ref_rule_discovery**). Em vez de utilizar LLMs apenas em tempo de execução, torna-se possível empregá-los para descobrir novos padrões de otimização a partir de consultas lentas e posteriormente destilar esse conhecimento em regras nativas de baixo

custo computacional.

Questões de segurança e privacidade também emergem como preocupação crescente (**ref_security_rag**). O uso de RAG frequentemente exige acesso a esquemas, metadados e informações operacionais potencialmente sensíveis, tornando necessária a investigação de técnicas de anonimização e ofuscação de esquema.

A literatura ainda aponta espaço para pesquisas envolvendo ajuste fino orientado ao domínio (*domain fine-tuning*) (**ref_domain_finetune**). Grande parte dos estudos atuais depende exclusivamente de *prompt engineering*, havendo pouca exploração do treinamento especializado com planos de execução, documentação de SGBDs e histórico de otimizações.

Por fim, observa-se uma carência de *benchmarks* dinâmicos (**ref_dynamic_benchmark**). Os conjuntos de avaliação predominantes permanecem estáticos e pouco representativos da variabilidade observada em ambientes corporativos reais. O desenvolvimento de estruturas experimentais capazes de incorporar deriva de dados, mudanças de *workload* e feedback contínuo constitui uma oportunidade relevante para futuras investigações.

3. Trabalhos Relacionados

Nesta seção, são analisados os principais estudos que utilizam Inteligência Artificial, especificamente LLMs e modelos aprendidos, para a reescrita e otimização de consultas SQL.

- **O uso da Inteligência Artificial na otimização de consultas em bancos de dados (Amaral, Andrade e Campos, 2025)**
  Os autores investigaram a viabilidade do uso do ChatGPT para a reescrita semântica de consultas em ambientes PostgreSQL. O estudo utilizou uma metodologia experimental com scripts Python para automatizar a execução e coleta de métricas, focando em transformações como o uso de *Common Table Expressions* (CTEs), pré-agregações e redução de subconsultas. Os resultados demonstraram que a intervenção da IA pode reduzir o tempo de execução em até 90% em cenários complexos com múltiplos *joins*, promovendo uma racionalização no uso de índices e tabelas.
- **LLM-R2: A Large Language Model Enhanced Rule-based Rewrite System (Li et al., 2024)**
  Este trabalho propõe o sistema LLM-R2, que utiliza LLMs para sugerir sequências de regras de reescrita para plataformas como o Apache Calcite. O diferencial técnico reside no uso de um modelo contrastivo treinado via currículo para selecionar demonstrações (*few-shot*) estruturalmente similares à consulta de entrada. Ao delegar a execução final das regras a um motor de banco de dados, o sistema garante executabilidade e equivalência semântica, superando abordagens de geração livre suscetíveis a alucinações.
- **R-Bot: An LLM-based Query Rewrite System (Sun et al., 2025)**
  O estudo introduz o R-Bot, um sistema que busca mitigar alucinações por meio de uma arquitetura baseada em *Retrieval-Augmented Generation* (RAG). O sistema utiliza recuperação híbrida, combinando características estruturais e semânticas, para buscar "receitas de reescrita" e evidências provenientes de fóruns técnicos e

manuais de SGBDs. O processo é refinado por um mecanismo de reescrita passo a passo com autorreflexão, permitindo que o modelo compreenda as inter-relações entre regras antes da aplicação final.

- **LITHE: A Query Rewrite Advisor using LLMs (Dharwada et al., 2025)**
  Os autores propõem o LITHE, sistema que realiza transformações diretamente no espaço de consulta (*SQL-to-SQL*) em vez do espaço de plano físico. A abordagem utiliza regras sensíveis ao banco de dados, fundamentadas em seletividade e metadados, combinadas com uma busca em árvore de Monte Carlo (*Monte Carlo Tree Search – MCTS*) guiada pelas probabilidades de *tokens* do LLM. Essa estratégia permite explorar múltiplos caminhos de reescrita quando a confiança do modelo é baixa, alcançando acelerações expressivas em comparação aos otimizadores nativos.
- **QUITE: A Query Rewrite System Beyond Rules with LLM Agents (Song et al., 2025)**
  Este trabalho apresenta o QUITE, um *framework* baseado em múltiplos agentes controlados por uma Máquina de Estados Finita (*Finite State Machine – FSM*). O sistema utiliza feedback em tempo real do banco de dados, obtido via comandos `EXPLAIN`, além de técnicas de injeção de *hints* para garantir que o plano otimizado seja seguido pelo motor de execução. Diferentemente de métodos puramente baseados em regras, o QUITE permite a descoberta de estratégias de otimização que não podem ser expressas por transformações fixas.

Diferente das abordagens de otimização interna, como Neo e Bao, que buscam substituir ou modificar componentes do otimizador do SGBD, e dos sistemas de reescrita baseados exclusivamente em regras ou focados apenas na redução isolada de latência, este trabalho propõe uma abordagem de reescrita de consultas SQL do tipo *SQL-to-SQL* baseada em Modelos de Linguagem de Grande Escala (LLMs), apoiada por mecanismos de validação e análise conduzidos pelo próprio sistema gerenciador de banco de dados.

O diferencial da proposta reside na busca por um equilíbrio entre desempenho e preservação semântica. Enquanto abordagens de geração livre oferecem maior flexibilidade de transformação, frequentemente sofrem com problemas de *semantic drift*, alucinações e regressões de desempenho, sistemas rigidamente baseados em regras limitam o espaço de otimização a transformações previamente codificadas. Neste contexto, o presente trabalho propõe uma camada externa e não invasiva de otimização, na qual o LLM atua como agente de reescrita semântica, enquanto o SGBD exerce o papel de mecanismo de validação e evidência técnica.

A abordagem proposta estrutura-se em múltiplos estágios de verificação. Inicialmente, a consulta reescrita é submetida à validação sintática por meio do *parser* do banco de dados. Em seguida, comandos como `EXPLAIN` e `EXPLAIN ANALYZE` são utilizados para analisar o plano de execução e comparar o custo estimado entre a consulta original e a versão otimizada. Por fim, a equivalência da transformação é avaliada por meio da comparação dos resultados produzidos pelas consultas, reduzindo o risco de alterações indevidas na semântica original.

Dessa forma, o sistema proposto estabelece um ciclo em que o LLM propõe transformações, o otimizador do SGBD fornece evidências de desempenho e os mecanismos de validação asseguram a correção da reescrita. Como resultado, busca-se desenvolver

uma solução de otimização prática, explicável e compatível com diferentes SGBDs relacionais, preservando a confiabilidade necessária para aplicações corporativas e ambientes reais de produção.

4. Materiais e Métodos

Tecnologias, instrumentos e procedimentos que serão usados no estudo. O Algoritmo 1 se refere ao método de ordenação `Bubblesort` expresso em linguagem `Python`.

Uma lista numérica:

1. Item 1
2. Item 2

Uma lista definida com letras sequenciais:

a) Item 1

b) Item 2

Algoritmo 1. Método de ordenação Bubblesort

```
1 def bubble_sort(alist):
2     for i in range(len(alist)-1,0,-1):
3         for j in range(i):
4             if alist[i]>alist[i+1]:
5                 temp = alist[i]
6                 alist[i] = alist[i+1]
7                 alist[i+1] = temp
```

5. Resultados e Discussão

Essa seção deverá ser escrita na segunda parte do trabalho, conhecida como TCC2, e deverá conter os resultados dos experimentos realizados, discussão comparando os resultados obtidos com outros encontrados em trabalhos similares, além de um parágrafo apontando as limitações da metodologia adotada.



Figura 1. Exemplo de uso de figura

Figura 2. Exemplo com sub-figuras



(a) Sub-figura 1



(b) Sub-figura 2

Tabela 2. Minha tabela

cabeçalho 1	**cabeçalho 2**
texto à esquerda	Existem muitas variações das passagens do Lorem Ipsum disponíveis, mas a maior parte sofreu alterações de alguma forma.

Tabela 3. Tabela com sub-tabelas

cabeçalho 1	**cabeçalho 2**
Contrary to popular belief, Lorem Ipsum is not simply random text. It has roots in a piece of classical Latin literature from 45 BC, making it over 2000 years old.	The standard chunk of Lorem Ipsum used since the 1500s is reproduced below for those interested.

(a) Sub-tabela da esquerda

cabeçalho 1	**cabeçalho 2**
Contrary to popular belief, Lorem Ipsum is not simply random text. It has roots in a piece of classical Latin literature from 45 BC, making it over 2000 years old.	The standard chunk of Lorem Ipsum used since the 1500s is reproduced below for those interested.

(b) Sub-tabela da direita

6. Considerações Finais

Essa seção deverá ser escrita na segunda parte do trabalho, conhecida como TCC2.